

Part 4.

메타데이터 필터링 — 검색 범위를 좁혀 정확도 높이기

메타데이터는 청크 본문이 아닌 부가 정보(파일명, 페이지 번호, 작성일, 카테고리 등)를 말한다. 벡터 유사도 검색만으로는 "어떤 문서에서 왔는가", "몇 페이지인가"를 구분할 수 없기 때문에, 메타데이터를 함께 저장하고 검색 시 필터로 활용하면 검색 정확도를 크게 높일 수 있다.

왜 필요한가

- **검색 범위 축소:** 여러 문서가 섞여 있을 때 특정 문서·특정 범위로 한정하면 무관한 청크가 결과에 끼어드는 것을 막는다.
- **의도 반영:** "A 매뉴얼의 환불 정책"처럼 사용자 질문에 문서 조건이 포함된 경우, 벡터 유사도만으로는 A와 B를 구분하기 어렵다.
- **다중 문서 관리:** 실전 RAG는 수십~수백 개 문서를 하나의 컬렉션에 넣는 경우가 많다. 메타데이터 없이는 출처 추적 자체가 불가능하다.

어떤 메타데이터를 넣는가

- **source:** 원본 파일명 (예: `"part3.pdf"`, `"faq.txt"`)
- **page:** PDF의 페이지 번호. 사용자에게 출처를 알려줄 때 필수적이다.
- **chunk_index:** 해당 문서에서 몇 번째 청크인지. 디버깅과 순서 복원에 유용하다.
- **category / document_type:** 문서의 종류 (예: `"policy"`, `"manual"`, `"faq"`). 질문 의도에 따라 문서 유형을 필터링할 수 있다.
- **date / created_at:** 작성일 또는 업데이트일. 최신 문서를 우선 검색하거나, 특정 기간의 문서만 조회할 때 쓴다.

메타데이터를 설계할 때는 *****"사용자가 어떤 조건으로 검색할 것인가"*****에서 거꾸로 생각한다. 사내 문서 RAG라면 `department`, `document_type`, `date`가 유용하고, 논문 RAG라면 `author`, `year`, `section`이 더 적합하다. 다만, 메타데이터가 너무 많으면 저장·관리 비용이 늘어나므로 필요한 것만 선별한다.

5주차 코드에서 메타데이터가 이미 쓰이고 있었다

5주차에서 ChromaDB에 청크를 저장할 때 `metadatas` 파라미터에 `source`와 `page`를 넣었다. 그러나 검색(`collection.query`)할 때는 필터 없이 유사도만으로 결과를 받아왔다. 즉, 메타데이터를 **저장은 했지만 활용하지 않은** 상태였다.

python

```
# 5주차 - 저장할 때 메타데이터를 넣었지만
collection.add(
    documents=chunks,
    metadatas=[{"source": "part3.pdf", "page": i} for i in range(len(chunks))],
    ids=[f"chunk_{i}" for i in range(len(chunks))]
)

# 검색할 때는 필터 없이 유사도만 사용
results = collection.query(query_texts=["환불 정책은?"], n_results=3)
```

이번 주에는 `where` 파라미터를 추가해 메타데이터 필터링을 적용한다.

ChromaDB where 필터 사용법

ChromaDB의 `query()` 메서드에 `where` 파라미터를 넘기면 메타데이터 조건에 맞는 청크만 검색 대상으로 삼는다.

기본 필터 — 특정 문서만 검색

python

```
results = collection.query(
    query_texts=["환불 정책은?"],
    n_results=3,
    where={"source": "part3.pdf"}
)
```

비교 연산자

연산자	의미	예시
<code>\$eq</code>	같다 (기본값)	<code>{"source": {"\$eq": "part3.pdf"}}</code>
<code>\$ne</code>	같지 않다	<code>{"source": {"\$ne": "old_doc.txt"}}</code>
<code>\$gt</code>	초과	<code>{"page": {"\$gt": 5}}</code>
<code>\$gte</code>	이상	<code>{"page": {"\$gte": 3}}</code>
<code>\$lt</code>	미만	<code>{"page": {"\$lt": 10}}</code>
<code>\$in</code>	목록 중 하나	<code>{"source": {"\$in": ["a.pdf", "b.pdf"]}}</code>

복합 필터 — 여러 조건 결합

python

```
results = collection.query(
    query_texts=["환불 정책은?"],
    n_results=3,
    where={
        "$and": [
            {"source": "part3.pdf"},
            {"page": {"$gte": 3}},
            {"page": {"$lt": 8}}
        ]
    }
)
```

```
]
}
)
```

`$and` 는 모든 조건을 동시에 만족하는 청크만 반환하고, `$or` 는 하나라도 만족하면 반환한다.

실습 — 필터 있는 검색 vs 없는 검색 비교

여러 PDF를 하나의 컬렉션에 저장한 상황을 만들고, 동일 질문에 대해 필터 유무에 따른 검색 결과를 비교한다.

1. 5주차에서 사용한 `part3.pdf` 와 새로운 PDF(예: `policy.pdf`)를 하나의 컬렉션에 함께 저장한다.
2. 필터 없이 검색하면 두 문서의 청크가 섞여서 돌아온다.
3. `where={"source": "part3.pdf"}` 를 추가하면 해당 문서의 청크만 결과에 나온다.

관찰 포인트

- 필터 없이 검색하면 질문과 무관한 문서의 청크가 상위에 올라오는 경우가 있다.
- 필터를 적용하면 검색 대상 자체가 줄어들어 관련 청크가 상위를 차지한다.
- 특히 유사한 주제를 다루는 문서가 여러 개일 때 효과가 두드러진다.

메타데이터 필터링의 한계

메타데이터 필터링은 강력하지만 만능은 아니다.

- **사용자가 조건을 명시해야 한다:** "A 문서에서 찾아줘"라고 해야 필터를 걸 수 있다. 사용자가 조건을 말하지 않으면 어떤 문서를 필터링해야 할지 시스템이 판단해야 하는데, 이것 자체가 또 다른 문제다.
- **메타데이터가 잘못되면 검색도 틀어진다:** 파싱 단계에서 페이지 번호를 잘못 매기거나 카테고리를 잘못 분류하면, 필터링이 오히려 정답 청크를 배제해 버린다.
- **필터가 너무 좁으면 결과가 없다:** `where` 조건을 지나치게 엄격하게 걸면 해당 조건을 만족하는 청크가 아예 없어서 빈 결과가 돌아올 수 있다.

"메타데이터는 검색의 1차 관문이다 — 벡터 유사도가 '무엇이 비슷한가'를 찾는다면, 메타데이터는 '어디에서 찾을 것인가'를 정한다."